Übungsaufgaben: Einfache Probleme

Algorithmen und Komplexität

Aufgabe 1: Kürzeste einfache Wege – nicht so "einfach"

Sei $G = (V, E)$ ein gerichteter Graph mit Kantengewichten $w\colon E \to \mathbb{R} \setminus \{0\}$. Die Länge eines Pfades ist die Summe seiner Kantengewichte. Ein Pfad heißt *einfach*, wenn er keinen Knoten wiederholt.

Intuitiv könnte man vermuten, dass jeder Teilpfad eines kürzesten einfachen Pfades ebenfalls ein kürzester einfacher Pfad zwischen seinen Endpunkten ist. Dies ist jedoch nicht immer der Fall.

(a) Geben Sie einen gerichteten, kantengewichteten Graphen G mit einem kürzesten einfachen v_0–v_k-Pfad $P = \langle v_0, v_1, \ldots, v_k \rangle$ an, der einen Teilpfad $\langle v_i, \ldots, v_j \rangle$ enthält, der *kein* kürzester einfacher v_i–v_j-Pfad in G ist.

Begründen Sie Ihre Antwort.

(b) Eine Gewichtsfunktion w heißt *konservativ*, wenn jeder gerichtete Kreis nichtnegatives Gesamtgewicht hat.

Zeigen Sie: Ist w konservativ und $P = \langle v_0, \ldots, v_k \rangle$ ein kürzester einfacher v_0–v_k-Pfad, so ist für alle $0 \le i \le j \le k$ auch $\langle v_i, \ldots, v_j \rangle$ ein kürzester einfacher v_i–v_j-Pfad.

Aufgabe 2: Jede Formel hat eine KNF

Zeigen Sie, dass zu jeder aussagenlogischen Formel φ über den Variablen $x_1, \ldots, x_n$ eine logisch äquivalente Formel ψ in konjunktiver Normalform (KNF) existiert.

Dabei dürfen keine zusätzlichen Variablen eingeführt werden. Die Konstruktion muss nicht effizient sein.

Hinweis: Sie dürfen die Wahrheitsbelegungen von φ betrachten. Überlegen Sie, wie man eine Klausel $C = (l_1, l_2, l_3)$ mit Literalen l_1, l_2, l_3 konstruiert die genau für die Belegung $(1, 0, 1)$ falsch ist.

Bemerkung. Die entstehende KNF kann exponentiell groß sein. Für SAT-Algorithmen verwendet man deshalb häufig die Tseytin-transformation, die in Polynomialzeit eine erfüllbarkeitsäquivalente KNF konstruiert, indem sie zusätzliche Variablen einführt.

Aufgabe 3: Die Party, zu der niemand kommen kann

Ada organisiert eine Party und möchte ihre drei Freunde Alice, Bob und Clara einladen.

Leider haben diese einige Wünsche:

1. Mindestens einer von Alice und Bob soll kommen.
2. Wenn Alice kommt, möchte sie Clara mitbringen.
3. Wenn Bob kommt, möchte er ebenfalls Clara mitbringen.
4. Clara kommt nur, wenn weder Alice noch Bob dabei ist.

(a) Formulieren Sie die Bedingungen als 2-SAT-Formel.
(b) Konstruieren Sie den Implikationsgraphen und bestimmen Sie seine starken Zusammenhangskomponenten.
(c) Kann Ada eine Gästeliste finden, die alle Wünsche erfüllt? Begründen Sie Ihre Antwort.
(d) Kann Ada die Party retten, indem sie genau einen der vier Wünsche ignoriert? Falls ja, welche Wünsche kommen dafür infrage? Geben Sie jeweils eine mögliche Gästeliste an.

Aufgabe 4: Es werde Licht!

In einem alten Labor befinden sich fünf Schalter A, B, C, D, E und sechs Lampen. Jeder Schalter kann entweder *ein-* oder *ausgeschaltet* sein.

Die Lampen sind etwas eigenwillig: Jede besitzt eine eigene Regel, wann sie leuchtet.

Lampe	Schalter	Die Lampe leuchtet genau dann, wenn ...
L_1	A, B	die beiden Schalter unterschiedlich stehen.
L_2	B, C	die beiden Schalter unterschiedlich stehen.
L_3	C, D	die beiden Schalter gleich stehen.
L_4	D, E	mindestens einer eingeschaltet ist.
L_5	A, E	nicht beide eingeschaltet sind.

(a) **Von Schaltern zu 2-SAT.**

Führen Sie für jeden Schalter $X \in \{A, B, C, D, E\}$ eine boolesche Variable x_X ein, wobei $x_X = 1$ bedeutet, dass der Schalter eingeschaltet ist.

Stellen Sie eine Formel φ in konjunktiver Normalform auf, die genau dann erfüllt ist, wenn alle fünf Lampen $L_1, \ldots, L_5$ leuchten.

Begründen Sie, weshalb φ eine 2-CNF-Formel ist.

(b) **Licht an!**

Konstruieren Sie den Implikationsgraphen G_φ und bestimmen Sie seine starken Zusammenhangskomponenten.

Entscheiden Sie mithilfe des 2-SAT-Algorithmus, ob alle fünf Lampen gleichzeitig leuchten können. Falls ja, bestimmen Sie aus den starken Zusammenhangskomponenten eine geeignete Schalterstellung.

(c) **Eine Lampe zu viel?**

Nun wird eine sechste Lampe L_6 eingebaut. Sie ist mit den Schaltern A und D verbunden und leuchtet genau dann, wenn diese *unterschiedlich* stehen.

Können jetzt alle sechs Lampen gleichzeitig leuchten?

Begründen Sie Ihre Antwort mithilfe des Implikationsgraphen.

Aufgabe 5: Der übereifrige Wanderer

Ein Wanderer plant eine Reise durch eine Region, deren Städte durch Straßen miteinander verbunden sind. Auf seiner Landkarte entsprechen die Städte den Knoten und die Straßen den Kanten eines einfachen, ungerichteten Graphen $G = (V, E)$.

(a) **Eine merkwürdige Beobachtung.**

Noch bevor er losläuft, fragt sich der Wanderer, ob es auf seiner Landkarte wohl immer zwei Städte gibt, an die gleich viele Straßen angrenzen. Gehen Sie davon aus, dass die Landkarte mindestens zwei Städte enthält.

Beweisen oder widerlegen Sie diese Aussage.

Nun wird es ernst: Der Wanderer möchte jede Straße *genau einmal* entlanggehen und anschließend zu seinem Ausgangspunkt zurückkehren.

Glücklicherweise weiß er, dass das Straßennetz zusammenhängend ist und einen Eulerkreis besitzt. Er muss ihn nur noch finden.

Dazu entwickelt er folgenden Algorithmus:

> *Starte bei einem beliebigen Knoten und durchlaufe wiederholt eine beliebige noch unbenutzte Kante. Sobald vom aktuellen Knoten keine unbenutzte Kante mehr ausgeht, beende den Algorithmus.*

(b) **Verlaufen trotz Eulertour?**

Finden Sie einen Graphen mit möglichst wenigen Knoten, der eine Eulertour besitzt, bei dem der vorgeschlagene Algorithmus jedoch scheitern kann.

Geben Sie eine konkrete Folge von Kanten an, die den Wanderer vorzeitig zum Stehen bringt.

(c) **Ein bisschen schlauer?**

Der Wanderer verbessert seinen Algorithmus:

> *Durchlaufe stets eine unbenutzte Kante und bevorzuge dabei Kanten zu bislang unbesuchten Knoten. Falls keine unbenutzte Kante mehr zur Verfügung steht, beende den Algorithmus.*

Ist der Algorithmus damit korrekt? Beweisen Sie Ihre Antwort oder geben Sie ein Gegenbeispiel an.

(d) **Wie macht man es richtig?**

Machen Sie sich mit dem Algorithmus von Hierholzer (Wikipedia) vertraut.

Versuchen Sie zu verstehen, wie dieser Algorithmus die Probleme der bisherigen Ansätze vermeidet.

Aufgabe 6: Zur Vorbereitung und Diskussion – Starke Zusammenhangskomponenten

In der Vorlesung haben wir gesehen, dass sich die Erfüllbarkeit einer 2-CNF-Formel mithilfe der starken Zusammenhangskomponenten (SZK) ihres Implikationsgraphen entscheiden lässt.

Doch wie können wir diese Komponenten effizient bestimmen?

Machen Sie sich mit dem *Algorithmus von Kosaraju* vertraut, beispielsweise anhand der Beschreibung auf Wikipedia (Kosaraju's Algorithm).

Beschäftigen Sie sich insbesondere mit folgenden Fragen:

1. Was ist die grundlegende Idee des Algorithmus? Welche Rolle spielen die beiden Tiefensuchen?
2. Warum werden die Kanten zwischen den beiden Tiefensuchen umgedreht?
3. Warum ist die Reihenfolge der Knoten bei der zweiten Tiefensuche entscheidend?
4. Können Sie den Algorithmus anhand eines kleinen Beispiels nachvollziehen?
5. Welche Laufzeit besitzt der Algorithmus?
6. Warum ist der Graph, der durch Kontraktion aller starken Zusammenhangskomponenten entsteht, stets ein DAG?